

VIETNAM NATIONAL UNIVERSITY, HO CHI MINH CITY
UNIVERSITY OF TECHNOLOGY
FACULTY OF COMPUTER SCIENCE AND ENGINEERING



Data Structure and Algorithm (CO2003)

Assignment

*Segment Tree
and its application
in Range Query*

Instructor(s): Nguyễn Đức Dũng

Students: Nguyễn Thái Học - 2311100 - TN01 Class

HO CHI MINH CITY, January 2025

Mục lục

1	Motivation and Idea	2
2	Structure	3
2.1	Building Segment Tree	4
3	Main operations	5
3.1	Updating	5
3.2	Querying	6
4	Evaluating the data structure	7
5	Application	7

1 Motivation and Idea

Imagine you are the owner of the sheep farm. Everyday, you leave the sheep going outside at the fields, and let your dog caring about them. However, in this field, there also contains wolves that may attack and kill the sheep. At the end of each day, you need to check how many sheep remain. Since you own at least a thousand sheep, so you don't want to count them individually until the midnight, making it sleepy as well.

To solve this problem, you come up with an idea: Hiring a team to count the sheep for you. The team divides the sheep into smaller groups, with each member responsible for counting one portion, and reporting the number on time. Once all the team members have finished, you simply add up their totals to calculate the final count. Additionally, you can count the sheep based on specific fields, herds, or characteristics, such as male, female, or immature sheep. This will give you the insight about the distribution and traits of your flock.



Hình 1: Obsessed by count sheep

The idea is quite similar to what Segment tree means. The example above provides a simple way to understand the core idea behind Segment Trees. In real world scenarios, there exists numerous problems where finding out the relationship between different data points with high frequency, which means you need to access a bunch of data points that closed to each others many time to get the insights about them. For instance, you might need to quickly determine the sum, maximum, or minimum value of a range in a dataset. Segment Trees are particularly useful in such cases, as they allow you to access and manipulate data efficiently, even when the dataset is large. Let's delve deeper into how this data structure works and why it is so effective for range queries and updates.

2 Structure

A Segment Tree is a versatile data structure that organizes the values of an original array in its leaf nodes and stores information about intervals (e.g., minimum, maximum, sum, product, or other advanced operations) in its internal nodes. It is particularly efficient for querying and updating values over intervals or segments of an array.

It is a full binary tree, which has each node either being a leaf node or having two children. The size of the original array determines the structure of the tree, with the total number of nodes being at most four times the size of the array. This allows the tree to be represented using a fixed-size array, enabling efficient data access in constant time.

```
Type in count: 12
Choose the operation (sum/mul/min/max)
By typing the name of the operation here: sum
Value of numbers for array:
0 : 35
1 : 8
2 : 5
3 : 3
4 : 51
5 : 14
6 : 21
7 : 10
8 : 37
9 : 6
10 : 31
11 : 19
#####
Original structure of sum Segment Tree :
          240
        116 124
       48  68 68 56
      43 5 54 14 31 37 37 19
     35 8 3 51 21 10 6 31
```

Hình 2: Visual representation of the Segment Tree

Above is the visual representation of the Segment Tree that i have build using C++. You can check the its implementation and the result of my work by typing the number of source nodes and the operation that you want to apply to the tree, it will generate random test cases, use 2 main operations and visualize it on the terminal . I just build it with simple operations. Hope you give feedback about my work for further improvements.

2.1 Building Segment Tree

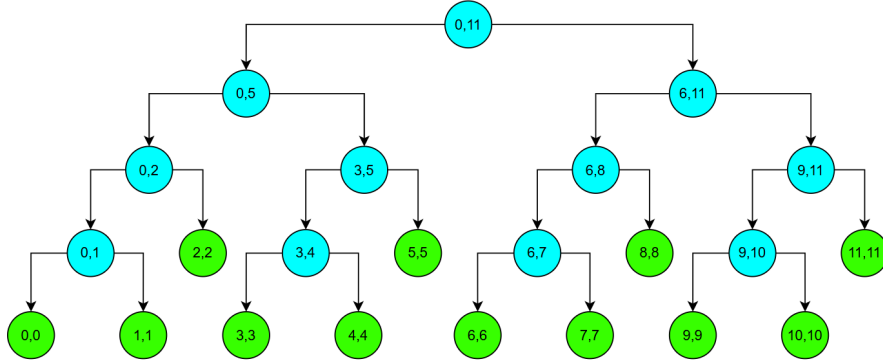
To construct the tree, we use the principle divide and conquer. Physically, we will begin with the leaf nodes from the source array. Each internal node then stores the result of an operation applied to its two child nodes. For example, in a sum Segment Tree, a parent node holds the sum of the values in its child nodes, while in a maximum Segment Tree, it holds the maximum value from its children.

Algorithmically, we will use the recursion to build the tree. The process starts with the root node at index 0 in the Segment Tree array. From there, we recursively move to its right child and left child by using the formula $2 \cdot \text{index} + 1$ and $2 \cdot \text{index} + 2$.

At each level, the range of the array $[n, n + 1, n + 2, \dots, m - 1]$ is divided into 2 smaller segments:

1. Left child handling the range $[n, n + 1, n + 2, \dots, m/2 - 1]$
2. Right child handling the range $[m/2, m/2 + 1, \dots, m - 1]$

Follow that principal, go through many nodes until reach the node that has the range $[x, x]$, where x is the index of that leaf node in the source array. After assigning leaf node's values, we backtrack to assign value for internal nodes. The internal nodes covers the range from 2 children, and carry the property about that range.



Hình 3: Indices for the segment

With the previous example, we can construct the array like below. The root node will at index 0, while other nodes follows to the formula to determine it positions. Some positions in this array may remain unused (null or empty) if their parent nodes correspond to elements in the source array that do not have children at the last level of the tree.

With n is the number of elements in source array, the Time complexity of the algorithm is $O(n)$ if the operation is at constant time. Space complexity is at most $4n$ so it equals to $O(n)$.

Source Array	0	1	2	3	4	5		
	35	8	5	3	51	14		
	6	7	8	9	10	11		
	21	10	37	6	31	19		
Segment Tree Array	0	1	2	3	4	5	6	7
	240	116	124	48	68	68	56	43
	8	9	10	11	12	13	14	15
	5	54	14	31	37	37	19	35
	16	17	18	19	20	21	22	23
	8	N/A	N/A	3	51	N/A	N/A	21
	24	25	26	27	28	29	30	
	10	N/A	N/A	6	31	N/A	N/A	

Hình 4: Representation of Source and Segement Tree Array

3 Main operations

One of the most powerful feature of the segment tree is its fast and efficient in querying and updating values when needed.

3.1 Updating

To change the value of a node that has index x in source array, we also use the recursion at the root node. The principle is quite similar to the building process. At each level, we move to either its left or right child according to the range that these child covers until reaching that leaf node that has range $[x, x]$. After updating the value, we use backtrack to change the value of parent nodes as its child's value changed until reach root node.

The time complexity of the algorithm is $O(\log_2(n))$ as equal to its height.

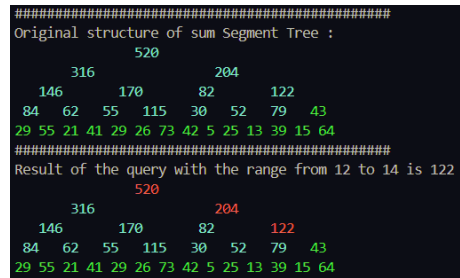
```
#####
Original structure of sum Segment Tree :
      499
    366   133
  216  150  104  29
127  89  52  98  25  79  17  12
60 67 26 63 47 5 29 69 8 17 34 45 9 8
#####
Updating at index 6 by value 14
      484
    351   133
  216  135  104  29
127  89  52  83  25  79  17  12
60 67 26 63 47 5 14 69 8 17 34 45 9 8
```

Hình 5: The path of recursion leading to the value changes

3.2 Querying

The query method in a Segment Tree uses recursion to retrieve values within the specified range. Conceptually, this process considers three distinct cases for each node:

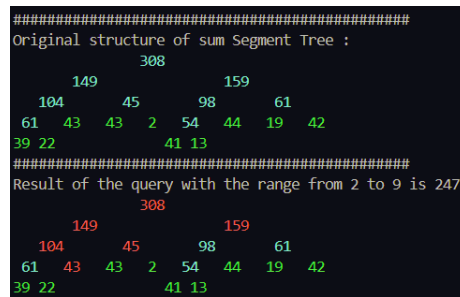
First, Node range is equal or within the range processed, we receive the value of this node and stop here. For example, in the below case, 122 hold the value for the range 12-14, equals to the range processed, it return that value until root node.



Hình 6: Querying case 1, 2

Second, Node range is invalid due to out of range processed, we call out invalid value so that the parent node can treat the it correctly. For instance, in sum operation, we return 0 as the value of the addition of 0 and any other value remains. In the above example, 82 was indeed processed, but it is out of the range processed, so it return 0 to its parent node.

Last, Node range partially cover the range processed, we just need to use the recursion to split that range. After receiving values from its children, we use the operation to find out the value of the corresponding range. In the case below, at node that contains value 308, 149, 104 meet this case.



Hình 7: Querying case 3

Time complexity: $O(\log n)$

4 Evaluating the data structure

Advantage:

1. High Performance: With segment tree, the querying and updating process all just need $O(\log n)$ time complexity, which is quite more effective compare to querying all elements in a simple array.
2. Flexibility: It has a usability in a wide range of problems, from calculating sum, defining maximum or minimum to searching.
3. Easy to build: Even though it seems hard to imagine at first, but having the fundamental in tree concepts will help you a lot in building this data structure. It can be easily visualized and build.

Disadvantage:

1. Space complexity: It takes 4 times bigger than the simple array to store the source and correlation value for intervals.
2. Hard to understand at first: Need to build strong foundation in tree data structure.
3. Not suitable for some problems: With the simple problems, you don't need to implement segment tree to solve them. But also consider the space complexity for large problems also.

5 Application

Segment Tree has many realistic application in programming and solving real world problems:

Array Management: Segment Trees efficiently manage segments of an array, particularly in problems requiring frequent queries and updates.

Summation and Search: They can be used to calculate sums, find the maximum or minimum values within a segment of an array, making them highly useful for data analysis and Range-based statistics problems.

Graphics Applications: In graphics, Segment Trees can manage and compute segments of images or geometric objects effectively based on its color, texture, and many other characteristics.

Optimization Segment Trees optimize problems related to scheduling, resource management, and operations involving strings, arrays, and data structures.

References

1. GeeksforGeeks. "Introduction to Segment Trees." Available at: <https://www.geeksforgeeks.org/introduction-to-segment-trees-2/>
2. CP-Algorithms. "Segment Tree - Construction and Applications." Available at: https://cp-algorithms.com/data_structures/segment_tree.html#construction
3. VNOI Wiki. "Cây Đoạn (Segment Tree) mở rộng." Available at: <https://wiki.vnoi.info/algo/data-structures/segment-tree-extend>
4. Wikipedia. "Segment Tree." Available at: https://en.wikipedia.org/wiki/Segment_tree
5. Viblo. "Cây Đoạn (Segment Tree) là gì?" Available at: <https://viblo.asia/p/cay-doan-segment-tree-la-gi-018J2KXeLYK>
6. YouTube. "Segment Tree Full Tutorial." Available at: <https://www.youtube.com/watch?v=27zXJ0mns1M&t=3559s>